Where the notes disagree

Where the notes disagree

Graal Analysis, revision 0.0.2.

These are the pairs the working notes do not settle. Each section quotes the claims and says what would settle the pair. This chapter does not choose a figure, and it does not average two of them.

A build that was never measured is left unmeasured in chapters 2 and 3. That absence is not a second number, so it is not repeated here.

Bushes, walls, and the passable lists

Four notes name the tiles of the 2 by 2 Bush stamp, and they do not describe the same test.

`formats/gobjset01.md` and `formats/tiletypes.txt` give ids 2, 3, 18, and 19 the type bytes 20, 20, 22, and 22. In the wall test those notes describe, a type of 20 or more is a wall, so those four ids are a wall. The stamp file itself has no collision column.

`formats/tile-walk.md` names the same four ids as the Bush stamp and then says they appear in the type-0 walk tables, and are therefore walkable when the sheet is type 0.

A lab capture of two 2.05 clients, written up in `protocol/packet-layouts.md`, names the tiles that grew back as grass as 2, 3, 18, and 19, and the tiles that grew back as bush as 34, 35, 50, and 51.

The id list 2, 3, 18, 19 is the one fact all three accounts share. The names "bush", "wall", "walkable", and "grass" are the disagreement.

The same notes give three different counts for what a type-0 sheet allows.

`formats/arrays-and-signatures.md`, from the 2.05 client, says array 0 of `arrays.dat` holds 934 tile ids and array 1 holds 116. In the old tile mode, a tile blocks unless it is in one of those two arrays. `[2.05]`

`formats/tile-walk.md` says one exported type-0 mask counted 3,195 non-blocking tiles and 682 blocking tiles. The same note says one extraction of the type-1 and type-2 rectangular grid counted 1,725 blocking cells and 2,371 walkable cells. Those sizes, 934 and 116, are not the 3,195 and 682 of the exported mask, and the notes never show that the two pairs are the same lists.

`formats/tiletypes.txt` says the two sorted allow lists were not in the decompile note it saw. The loader it documents keeps a wall test and a water test: a type of 20 or more is a wall, a type of exactly 11 is water, and types the official list called special stay walkable when they are under 20. The same note says the official client splits the tests: throw handling does not stop on type 20, while a shape type above 19 blocks the body. Types 20, 21, and 22 are all walls in the loader that note documents.

The 2.05 walking routine in chapter 2 blocks a tile that is in neither passable list (`client-internals/decomp-findings-2026-10-25-m/01-walking-rule.md`). It does not publish the length of those lists.

What would settle it is one 2.05 session that records, for ids 2, 3, 18, 19 and for ids 34, 35, 50, 51, the type byte, whether each id sits in array 0 or array 1, and which ids grow back after a cut, with the sheet type taken from that

same client. The three counts settle when a note shows they are the same lists, different sheets, or different tests. Nothing in the present notes shows that.

Knockback in 1.39

`version-profiles/matrix/rules-139.md` records two distances for one hurt.

The program: after the counter drops, while it is strictly greater than 34, readings 39 down to 35, the player moves one tile along the stored vector. That window is five steps. `[bin]`

The offline touch test measured four tiles of knockback. `[lab]`

The same note says the four-tile figure may have lost a step to collision or to a one-tick error in the video, and it calls the binary's five steps the figure to copy unless a re-run shows otherwise. That sentence is the note's preference. It is not a fifth step observed on the video, and this book does not adopt it as the distance.

What would settle it is a new offline run of 1.39 on open ground, with the counter read each tick or with a frame-accurate recording at the 50 ms frame, counting any step the wall test refused.

The first Graal's sword knockback is a different gap. The export mishandles the length of the delta, so a unit length is not tagged at all (`version-profiles/matrix/rules-1st-graal.md`). That is an unread number, not a second measurement of the 1.39 video.

Listserver shapes

Three notes describe the directory login, and they do not use the same words for the bytes.

`protocol/login-frames-by-version-2026-10-25.md`, quoted in chapter 1, says:

- 2.00 and 2.05 send one plain zlib frame: list id 0 and the list name `newmain`, then they close. No account, no password, no token, no key. The built-in list port in those notes is 14921.
- 2.17 sends one plain zlib frame with two messages: id 0 and `newmain`, then id 1 with the account and the password. No token and no cipher. The port named there is 14922.
- 2.22 and 2.31 send a first plain zlib frame of list id 7, one key byte, the token `GNW30123`, and `newmain`, then a second frame in generation 5 carrying id 1.

`tools/classic-tools/serverlist3.md` describes a different program, the Server Lister ELF, and says a client has two useful lines. An edition line selects directory rows whose edition contains that text and whose last write is under ten minutes. An account line loads the account by name. The password is checked locally, as a hash of the first eight characters. Any other line is ignored. "There is no later cipher and no version token." `[bin]`

`server-notes/lab-servers-2026-10-30.md` records what a loopback listserver version 2 showed three real clients:

Client	What it asked	What it got
2.05 rev 2	The older client port, edition only	A list, and then it reached the game
2.17.1	The later client port, account and password in the plain form	The Classic tab, and then the game
2.22	The same later port, the generation that 2.22 uses	Bronze, a status line, Show More, Donations, and then the game

The lab note adds that this listserver chooses its reply by login type and does not check the version token.

"Edition only" and "list id 0 and `newmain`" may be one observation in two vocabularies. "No version token" on the Server Lister ELF and "the client sends `GNW30123`" may be two programs. The notes have not laid the bytes of one capture beside both descriptions, so this book does not merge them into one layout.

The same Server Lister note gives the game server a different hello on the server port: a kind and an eight-byte token, `GLSER001` or `GLSER002`, then the world name. The client-frame note does not describe that hello. It is included here so it is not folded into `GNW30123`.

What would settle it is one loopback capture for 2.05, one for 2.17, and one for 2.22, each written out as the first frames, set beside the edition line and the account line and beside the three shapes above, until each frame is shown to be one of those layouts or a further one.

How many bits a Z3 code uses

`formats/level-binary.md` says `Z3-V1.02` uses 11-bit codes, and says `Z3-V1.03` and `Z3-V1.04` use 12. Only the 1998 Java Zelda Online games read the `.02` file, in that note. The converter reads `Z3-V1.02` as 11-bit codes (`tools/level-convert/README.md`).

`formats/level-binary-client.md` says the 2.05 and 2.31 readers use 12 bits for both `Z3-V1.*` versions they handle. It does not mention `Z3-V1.02`.

Read about `.03` and `.04`, the two notes agree on 12 bits. Read as a width for every Z3 file, they do not: 11 in the server note, 12 in the sentence that says "both" versions. The client note's pair is the pair in the 2.05 string table, which lists `Z3-V1.03` and `Z3-V1.04` and does not list `Z3-V1.02`.

What would settle the unscoped reading is a `Z3-V1.02` file decoded both ways, and a statement of whether 2.05's reader is ever called on that magic. The present client note does not make that call.

Whole tiles and fractions

`formats/level-binary.md` says the coordinates in the binary records are whole tiles. `[srv]` The same note later says `Z3-V1.03` link numbers are floating point, including halves, and gives a box and an arrival that include `43.5`.

`formats/glevnw01.txt` allows fractional NPC positions. The converter's example of a value cut off on the way into GR-V1 is `10.5` (`tools/level-convert/README.md`). The arrival count in `client-internals/decomp-findings-2026-10-25-f/02-link-arrival-expressions.md` includes halves such as `24.5`: of 18,710 arrival fields, 12,035 were plain numbers, integers and halves together.

The notes do not say which fields the whole-tile sentence covers.

What would settle it is a field list: for each of link box, link arrival, baddy, NPC, chest, and sign, the note that states the type, with one real file of each magic beside it. Until that list exists, the whole-tile sentence and the fractional fields both stand.

Where a sign list ends

`formats/level-binary-client.md` says 2.05 reads signs through to the end of the file. The same note says 2.31 stops at the first empty line, because a height block can follow.

`formats/level-binary.md` says the sign section runs until an empty line or the end of the file.

Both client behaviours fit that sentence. They do not fit each other when a file contains a blank line and a `HEIGHTS` block. The server layout does not say which bytes are signs in that file.

What would settle it is one `GR-V1.03` or `GR-V1.04` file that has signs, a blank line, and a `HEIGHTS` block, read by the 2.05 reader, the 2.31 reader, and the server reader, with the sign count each one keeps.

An offset after playerx

Real link fields contain `playery+10`, `playerx-15`, and `playery-1.5`. Across the archive counted in `client-internals/decomp-findings-2026-10-25-f/02-link-arrival-expressions.md`, 21 arrivals in 14 levels added a signed number. The note leaves open whether the stock client evaluates the offset, reads it as zero, or keeps only the keyword. No capture shows it. The converter keeps the spelling and does not answer the question (`tools/level-convert/README.md`).

A negative arrival in `Z3-V1.03`, which means the player's own coordinate is kept (`formats/level-binary.md`), is a different convention. Neither note says one client accepts the other spelling.

What would settle the offset is a capture, or a traced evaluation, of one link whose arrival is `playerx` plus a signed number, with the arrival coordinate the client actually uses.

What GENERATED names, and where HEIGHTS lives

`formats/grmap001.txt` records three writers.

The dungeon generator sets `GENERATED` to the bottom-right level name. In that file `GENERATED` is a filename, and there is no `LEVELNAMES` block.

The terrain generator writes `GENERATED 1`, and the `1` is a flag. Heights in that writer's level files are a 9 by 9 `HEIGHTS` block. The height field on the gmap is `HEIGHTMAP`.

GraalLevelGen writes a `LEVELNAMES` list of quoted filenames. `GENERATED` does not mean the same thing in all three files.

The server reader stores `GENERATED` and does not use it to rebuild names. `[srv]` (`formats/gmap.md`).

`protocol/format-notes-from-salvage.md` calls a gmap "a level list that carries HEIGHTS blocks and is not itself a board." The second half, that a gmap is not a board, matches the reader notes. The first half is coarser than they are: in the detailed notes `HEIGHTS` belongs to the level, `HEIGHTMAP` belongs to one writer, and a server can read a gmap that has neither.

What would settle the salvage sentence is a line in that note which says whether "HEIGHTS blocks" means the level's `HEIGHTS`, the gmap's `HEIGHTMAP`, or both. What would settle `GENERATED` is already the three files. They are three meanings. A single glossary entry that picks one of them would be a fourth claim.

Which board GraalEditor writes

`formats/glevnw01.txt` says GraalEditor's `BOARD 0` is the tile layer as 64 lines of 64 decimal indices, not a base64 run, and that this shape is not what the level loader accepts.

`tools/graaleditor.md` says an earlier note says GraalEditor writes a board as decimal tile indices. The same tools index warns that this conflicts with the base64 rows the built-in 2.05 and 2.22 editors write, and that it has not been re-checked against a file GraalEditor itself saved.

The built-in editors and GraalEditor 2.3.1.1 are different programs in both notes. The open point is the file GraalEditor saves.

What would settle it is one level saved from GraalEditor 2.3.1.1, with the magic and the board lines read back. Until that file is in the notes, the decimal grid and the "not re-checked" warning both stand.

A sketch that still prints another frame

`protocol/PROTOCOL-CHEATSHEET.md` still shows a raw byte after a 7-or-8-byte version, and it still says RC clients do not scramble. It names packets 100, 101, and 102 as a file length, tiles, and a file blob. It says the sender cuts frames into 1,024-byte slices.

`protocol/login-and-walk.md` and `protocol/codec-and-framing.md` describe the login with no mystery byte. The codec note says the extra byte is the server's view of generation 3, that RC 1.x is generation 3 as well, and that only the web client is unscrambled. The packet notes name 100, 101, and 102 as raw-data, board, and file. The codec note does not give a 1,024-byte slice.

`protocol/login-recovery-start.md` still shows a guessed login id of 1, a level-request id of 16, and a player-update id of 32. `protocol/login-and-walk.md` tells the reader to drop those ids. In that note the login is type 0, the level request is client packet 0, and a walk is client packet 2.

`protocol/login-and-walk.md` also withdraws an older direction order, south, west, east, north. The order that note keeps, and the order chapter 1 uses, is 0 up, 1 left, 2 down, 3 right. The note says this matches the 2.22 command text and the server.

These are withdrawals and replacements already written in the later notes. This book follows the later notes for the session in chapter 1 because that is where a complete layout is written down. It does not treat the sketch as disproved by the act of being older.

What would settle each leftover line is a capture set beside it: a 2.05 login for the mystery byte and the guessed ids, a generation-3 RC session for the cipher, a generation-5 flush for the slice size, and four labelled facings for the direction order.

Two further withdrawals sit in the notes and are not reopened here, because the same note that stated them has already replaced them. An earlier version of `formats/gmap.md` said 2.05 had no `GRMAP001` string. The current note says the string is present and that no caller was found. An earlier sign note described a second alphabet. `formats/sign-text.md` withdraws it and keeps one alphabet, shared by the 2.05 client and the server. An older summary called a GR-V1 board a 64 by 64 array of 2-byte tiles. `formats/level-binary.md` withdraws that for the file. Packet 101, in chapter 1, is 8,192 bytes of 16-bit tile ids on the wire. What would put the 2-byte sentence back onto the file is a `GR-V1.03` whose board is those 8,192 raw bytes. The re-encode of 144 cached boards, cited in chapter 1, is the check the file note already records.